

Defensive Operations Guide

How to use the Security Monitoring Dashboard to **protect** a host (your VPS, home server, or family computers). This covers monitoring, intrusion detection, IP blocking, firewall management, and alerting.

Overview tab — your situational awareness

The **Overview** is your at-a-glance health screen, refreshed every 5 seconds:

- **Active Connections** — current established TCP connections.
- **Listening Ports** — services accepting connections (review these regularly; unexpected listeners can indicate malware or misconfiguration).
- **Blocked IPs / Banned (IDS)** — current enforcement counts.
- **Unack Alerts** — security events awaiting your review.
- **Network Throughput** — live recv/sent chart; sudden spikes can signal exfiltration, DDoS, or a busy service.
- **System Status & Service Health** — CPU/memory/disk + whether `ufw`, `fail2ban`, and `ssh` are active.
- **Top Talkers** — remote IPs with the most connections, each with a one-click **Block** button.

 **Daily habit:** glance at listening ports and top talkers. Anything you don't recognize is worth investigating.

Connections tab — deep network visibility

Lists every tracked connection (proto, local/remote address, status, PID, process). Use the filter box to search by IP, port, or process name.

Workflow — investigating a suspicious connection:

1. Filter by the process name or remote IP.
2. Note the PID and process — confirm it's a service you expect.
3. If it's malicious/abusive, click **Block** on an ESTABLISHED row to firewall the remote IP instantly.

For full visibility (including other users' processes) run the dashboard as root / via the systemd service.

Firewall tab — blocking bad actors

The firewall manager supports **ufw** or **iptables** backends and keeps a **persistent block list** in `data/blocked_ips.json` (survives restarts).

DRY-RUN vs enforcing

- **DRY-RUN** (`firewall.enforce: false` , default): blocks are recorded and logged, and the exact command that would run is written to the log — but the system firewall is **not** touched. Perfect for testing safely.
- **Enforcing** (`firewall.enforce: true`): blocks apply real `ufw` / `iptables` rules. Requires root.

Block an IP

1. Go to **Firewall**.
2. Enter the IP and an optional reason → **Block IP**.
3. The IP appears in the block list with who/when/why. Click **Unblock** to remove.

Re-applying blocks after reboot

When enforcing with iptables, call `firewall_manager.reapply_all()` on boot, or simply restart the service — `ufw` rules persist natively. The block list itself is always preserved on disk.

Automatic blocking

When a single remote IP exceeds `monitoring.connection_threshold` established connections, a **high-severity alert** fires automatically. Review it on the Alerts tab and block with one click, or wire it to fail2ban for fully automated banning (see below).

IDS / fail2ban tab — intrusion detection

If `fail2ban` is installed, the dashboard shows each **jail** with currently failed/banned counts and the list of banned IPs. You can **ban** or **unban** IPs per jail directly from the UI.

If fail2ban is not present, a **built-in SSH brute-force watcher** tails `/var/log/auth.log` (or `/var/log/secure`) and raises a high-severity alert after repeated failed SSH logins from the same IP.

Recommended fail2ban setup

```
sudo cp config/fail2ban-sshd.local.example /etc/fail2ban/jail.local
# review bantime / findtime / maxretry, then:
sudo systemctl restart fail2ban
```

The example jail bans an IP for 1 hour after 5 failures in 10 minutes and uses `ufw` as the ban action — aligning fail2ban with the dashboard's firewall.

Alerts tab — never miss an event

Alerts are severity-rated: **info**, **low**, **medium**, **high**, **critical**. Sources include the network monitor, firewall, IDS, scanner, and auth (failed logins).

- Filter by minimum severity.
- **Ack** an alert once you've reviewed it (clears it from the unacknowledged count).
- **Webhook**: set `alerts.webhook_url` (Slack/Discord/generic) and `alerts.webhook_min_severity` to get pushed notifications for serious events.

All alerts are also written to `logs/alerts.log` (JSON lines) for retention and SIEM ingestion.

Logs & audit trail

File	Contents
<code>logs/app.log</code>	Application/runtime log (rotating).
<code>logs/alerts.log</code>	Every alert as JSON lines.
<code>logs/audit.log</code>	Accountability trail — every login, block/unblock, ban, and scan with actor + timestamp.

Ship these to your central logging/SIEM for long-term retention.

Example defensive scenario — “My VPS is under SSH attack”

1. **Alerts** shows “Repeated SSH auth failures from 198.51.100.7” (high).
2. Open **IDS / fail2ban** — the `sshd` jail already banned the IP (if fail2ban is configured). If not, go to **Firewall** and block it manually.
3. Check **Connections** to confirm no established session from that IP.
4. Harden: ensure SSH uses keys only (`PasswordAuthentication no`), and that `ufw allow OpenSSH` + `ufw enable` are set.
5. **Ack** the alert once handled.

See [BEST_PRACTICES.md](#) (BEST_PRACTICES.md) for hardening checklists and more scenarios (protecting family computers, monitoring a home network, etc.).